

Autoencoderi

Nikola Mamić, Jakov Tušek, Petar Matejčić

- Učenje reprezentacija podataka

Osnovna arhitektura autoenkodera

- Učenje reprezentacija podataka
- Cilj: što bolja rekonstrukcija ulaza

Osnovna arhitektura autoenkodera

- Učenje reprezentacija podataka
- Cilj: što bolja rekonstrukcija ulaza
- Enkoder: $\phi : \mathbb{R}^d \rightarrow \mathbb{R}^p$

Osnovna arhitektura autoenkodera

- Učenje reprezentacija podataka
- Cilj: što bolja rekonstrukcija ulaza
- Enkoder: $\phi : \mathbb{R}^d \rightarrow \mathbb{R}^p$
- Dekoder: $\psi : \mathbb{R}^p \rightarrow \mathbb{R}^d$

Osnovna arhitektura autoenkodera

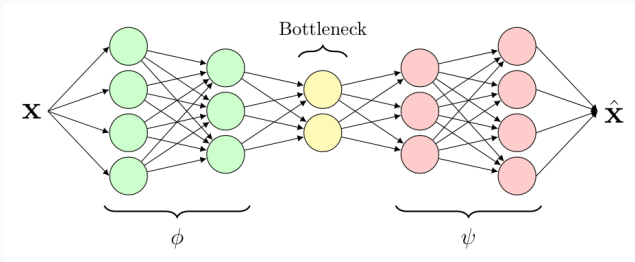
- Učenje reprezentacija podataka
- Cilj: što bolja rekonstrukcija ulaza
- Enkoder: $\phi : \mathbb{R}^d \rightarrow \mathbb{R}^p$
- Dekoder: $\psi : \mathbb{R}^p \rightarrow \mathbb{R}^d$

$$\hat{\mathbf{x}} = \psi(\phi(\mathbf{x}))$$

Osnovna arhitektura autoenkodera

- Učenje reprezentacija podataka
- Cilj: što bolja rekonstrukcija ulaza
- Enkoder: $\phi : \mathbb{R}^d \rightarrow \mathbb{R}^p$
- Dekoder: $\psi : \mathbb{R}^p \rightarrow \mathbb{R}^d$

$$\hat{\mathbf{x}} = \psi(\phi(\mathbf{x}))$$



- Mjerimo razliku između originala $\mathbf{x}$ i rekonstrukcije $\hat{\mathbf{x}}$

Funkcije gubitka

- Mjerimo razliku između originala $\mathbf{x}$ i rekonstrukcije $\hat{\mathbf{x}}$
- MSE za realne podatke:

$$L(\phi, \psi) = \frac{1}{n} \sum_{i=1}^n \|x^{(i)} - \psi(\phi(x^{(i)}))\|^2$$

Funkcije gubitka

- Mjerimo razliku između originala $\mathbf{x}$ i rekonstrukcije $\hat{\mathbf{x}}$
- MSE za realne podatke:

$$L(\phi, \psi) = \frac{1}{n} \sum_{i=1}^n \|x^{(i)} - \psi(\phi(x^{(i)}))\|^2$$

- *Cross-entropy* za binarne podatke:

$$L(\phi, \psi) = -\frac{1}{n} \sum_{i=1}^n \left[x^{(i)} \log(\hat{x}^{(i)}) + (1 - x^{(i)}) \log(1 - \hat{x}^{(i)}) \right]$$

Denoising Autoencoders

- Cilj: uklanjanje šuma u podacima

Denoising Autoencoders

- Cilj: uklanjanje šuma u podacima
- *Bottleneck*: sažimanje reprezentacije

Denoising Autoencoders

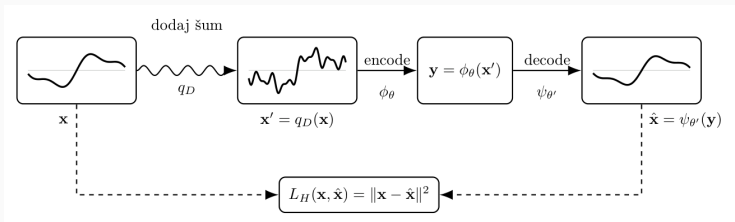
- Cilj: uklanjanje šuma u podacima
- *Bottleneck*: sažimanje reprezentacije
- *Zadržavanje bitnih značajki podataka + Odbacivanje slučajnih smetnji*

Denoising Autoencoders

- Cilj: uklanjanje šuma u podacima
- *Bottleneck*: sažimanje reprezentacije
- *Zadržavanje bitnih značajki podataka + Odbacivanje slučajnih smetnji*
- Zašumljeni ulaz $\rightarrow$ čista rekonstrukcija

Denoising Autoencoders

- Cilj: uklanjanje šuma u podacima
- *Bottleneck*: sažimanje reprezentacije
- *Zadržavanje bitnih značajki podataka* + *Odbacivanje slučajnih smetnji*
- Zašumljeni ulaz $\rightarrow$ čista rekonstrukcija



- Cilj: povećati **rijetkost** reprezentacije u *bottleneck* sloju

- Cilj: povećati **rijetkost** reprezentacije u *bottleneck* sloju
- Neuron je "aktivan" ako mu je izlaz velik, "neaktivan" ako je blizu nule

- Cilj: povećati **rijetkost** reprezentacije u *bottleneck* sloju
- Neuron je "aktivan" ako mu je izlaz velik, "neaktivan" ako je blizu nule
- Prosječna aktivacija j -tog neurona *bottleneck* sloja:

$$\hat{\rho}_j = \frac{1}{m} \sum_{i=1}^m \left[a_j^{(R)}(x^{(i)}) \right]$$

- Cilj: povećati **rijetkost** reprezentacije u *bottleneck* sloju
- Neuron je "aktivan" ako mu je izlaz velik, "neaktivan" ako je blizu nule
- Prosječna aktivacija j -tog neurona *bottleneck* sloja:

$$\hat{\rho}_j = \frac{1}{m} \sum_{i=1}^m \left[a_j^{(R)}(x^{(i)}) \right]$$

- Želimo postići $\hat{\rho}_j = \rho$, gdje je ρ parametar rijetkosti (npr. $\rho = 0.05$)

- Dodajemo penalizacijski sumand funkciji gubitka:

$$\sum_{j=1}^{R_k} \text{KL}(\rho || \hat{\rho}_j) = \sum_{j=1}^{R_k} \rho \log \frac{\rho}{\hat{\rho}_j} + (1 - \rho) \log \frac{1 - \rho}{1 - \hat{\rho}_j}$$

- Dodajemo penalizacijski sumand funkciji gubitka:

$$\sum_{j=1}^{R_k} \text{KL}(\rho || \hat{\rho}_j) = \sum_{j=1}^{R_k} \rho \log \frac{\rho}{\hat{\rho}_j} + (1 - \rho) \log \frac{1 - \rho}{1 - \hat{\rho}_j}$$

- Tretiramo aktivaciju neurona kao Bernoullijevu slučajnu varijablu s empirijskim očekivanjem $\hat{\rho}_j$

Sparse Autoencoder — Funkcija gubitka

- Dodajemo penalizacijski sumand funkciji gubitka:

$$\sum_{j=1}^{R_k} \text{KL}(\rho || \hat{\rho}_j) = \sum_{j=1}^{R_k} \rho \log \frac{\rho}{\hat{\rho}_j} + (1 - \rho) \log \frac{1 - \rho}{1 - \hat{\rho}_j}$$

- Tretiramo aktivaciju neurona kao Bernoullijevu slučajnu varijablu s empirijskim očekivanjem $\hat{\rho}_j$
- Nova funkcija gubitka:

$$C_{\text{sparse}}(W, b) = C(W, b) + \beta \sum_{j=1}^{R_k} \text{KL}(\rho || \hat{\rho}_j)$$

Sparse Autoencoder — Funkcija gubitka

- Dodajemo penalizacijski sumand funkciji gubitka:

$$\sum_{j=1}^{R_k} \text{KL}(\rho || \hat{\rho}_j) = \sum_{j=1}^{R_k} \rho \log \frac{\rho}{\hat{\rho}_j} + (1 - \rho) \log \frac{1 - \rho}{1 - \hat{\rho}_j}$$

- Tretiramo aktivaciju neurona kao Bernoullijevu slučajnu varijablu s empirijskim očekivanjem $\hat{\rho}_j$
- Nova funkcija gubitka:

$$C_{\text{sparse}}(W, b) = C(W, b) + \beta \sum_{j=1}^{R_k} \text{KL}(\rho || \hat{\rho}_j)$$

- $\beta = 0 \Rightarrow$ jednak osnovnom modelu

Sparse Autoencoder — Backpropagation

- Jedina promjena od osnovnog modela: delta *bottleneck* sloja

Sparse Autoencoder — Backpropagation

- Jedina promjena od osnovnog modela: delta *bottleneck* sloja
- Standardni model:

$$\delta_i^{(R)} = \left(\sum_j W_{ji}^{(R)} \delta_j^{(R+1)} \right) f'(z_i^{(R)})$$

Sparse Autoencoder — Backpropagation

- Jedina promjena od osnovnog modela: delta *bottleneck* sloja
- Standardni model:

$$\delta_i^{(R)} = \left(\sum_j W_{ji}^{(R)} \delta_j^{(R+1)} \right) f'(z_i^{(R)})$$

- Sparse model:

$$\delta_i^{(R)} = \left(\left(\sum_j W_{ji}^{(R)} \delta_j^{(R+1)} \right) + \beta \left(-\frac{\rho}{\hat{\rho}_i} + \frac{1-\rho}{1-\hat{\rho}_i} \right) \right) f'(z_i^{(R)})$$

Sparse Autoencoder — Backpropagation

- Jedina promjena od osnovnog modela: delta *bottleneck* sloja
- Standardni model:

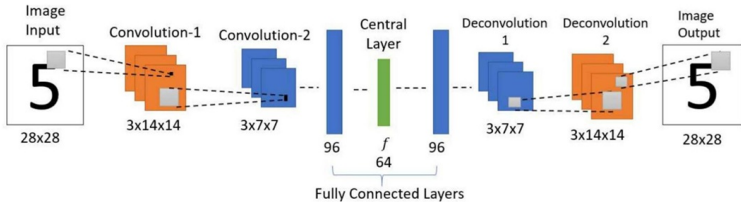
$$\delta_i^{(R)} = \left(\sum_j W_{ji}^{(R)} \delta_j^{(R+1)} \right) f'(z_i^{(R)})$$

- Sparse model:

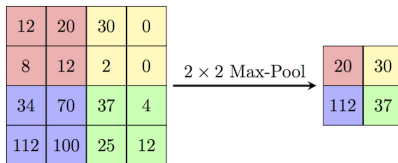
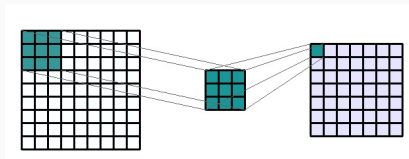
$$\delta_i^{(R)} = \left(\left(\sum_j W_{ji}^{(R)} \delta_j^{(R+1)} \right) + \beta \left(-\frac{\rho}{\hat{\rho}_i} + \frac{1-\rho}{1-\hat{\rho}_i} \right) \right) f'(z_i^{(R)})$$

- Prije svake epohe: jedan prolaz po svim podacima za izračun $\hat{\rho}$

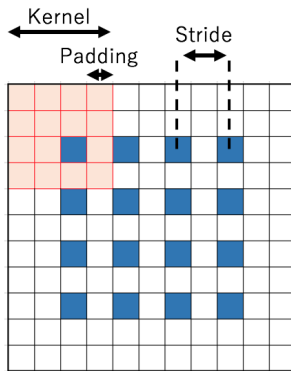
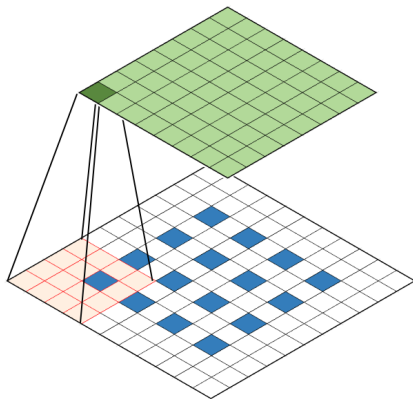
Konvolucijski autoenkoder



Konvolucijski sloj



Transponirana konvolucija



Kernel: 4
Stride: 2
Padding: 1

Uklanjanje šuma iz zvuka

- Audio zapis kao vektor: $x \in \mathbb{R}^{2048}$

Uklanjanje šuma iz zvuka

- Audio zapis kao vektor: $x \in \mathbb{R}^{2048}$
- Arhitektura:

$$2048 \rightarrow 256 \rightarrow 64 \rightarrow 256 \rightarrow 2048$$

Uklanjanje šuma iz zvuka

- Audio zapis kao vektor: $x \in \mathbb{R}^{2048}$
- Arhitektura:

$$2048 \rightarrow 256 \rightarrow 64 \rightarrow 256 \rightarrow 2048$$

- Aktivacija: $\tanh$

Uklanjanje šuma iz zvuka

- Audio zapis kao vektor: $x \in \mathbb{R}^{2048}$
- Arhitektura:

$$2048 \rightarrow 256 \rightarrow 64 \rightarrow 256 \rightarrow 2048$$

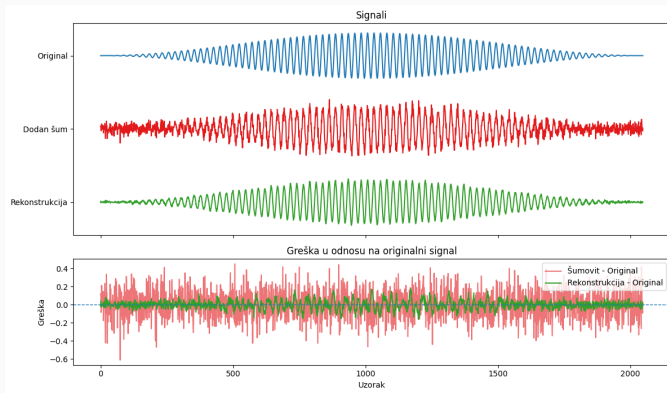
- Aktivacija: $\tanh$
- Primjena na nasumičnom šumu

Uklanjanje šuma iz zvuka

- Audio zapis kao vektor: $x \in \mathbb{R}^{2048}$
- Arhitektura:

$$2048 \rightarrow 256 \rightarrow 64 \rightarrow 256 \rightarrow 2048$$

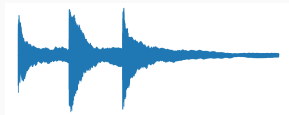
- Aktivacija: $\tanh$
- Primjena na nasumičnom šumu



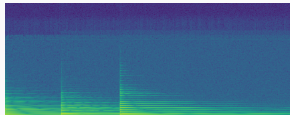
- Zvuk ima vremensku strukturu

Spektrogramski pristup

- Zvuk ima vremensku strukturu
- STFT: audio signal $\rightarrow$ spektrogram

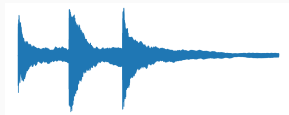


STFT $\rightarrow$

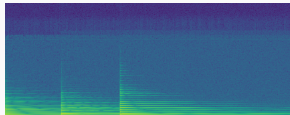


Spektrogramski pristup

- Zvuk ima vremensku strukturu
- STFT: audio signal $\rightarrow$ spektrogram
- Primjer: 16000 uzoraka $\rightarrow 256 \times 128$ matrica

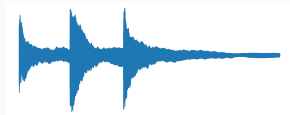


STFT $\rightarrow$

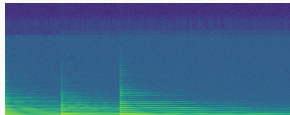


Spektrogramski pristup

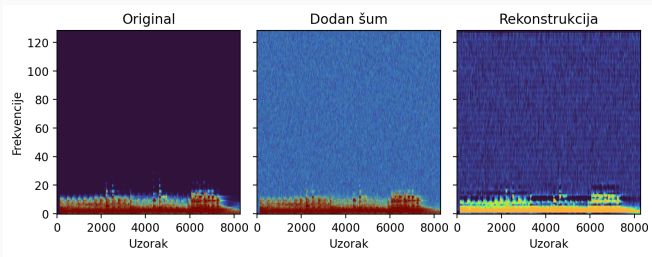
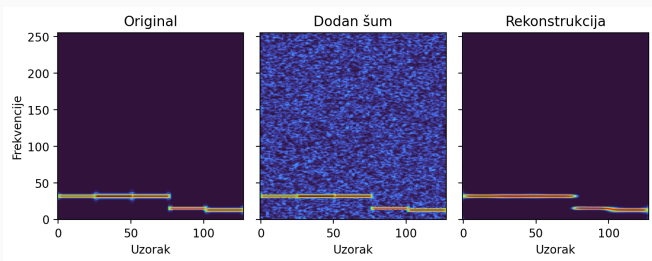
- Zvuk ima vremensku strukturu
- STFT: audio signal $\rightarrow$ spektrogram
- Primjer: 16000 uzoraka $\rightarrow 256 \times 128$ matrica
- Uklanjanje šuma iz zvuka $\rightarrow$ uklanjanje šuma iz slike



STFT $\rightarrow$



Rezultat na spektrogramu



- NSL-KDD: $> 120\,000$ zapisa TCP veza, 125 atributa

- NSL-KDD: $> 120\,000$ zapisa TCP veza, 125 atributa
- $\sim 50\%$ normalan promet, $\sim 50\%$ napadi

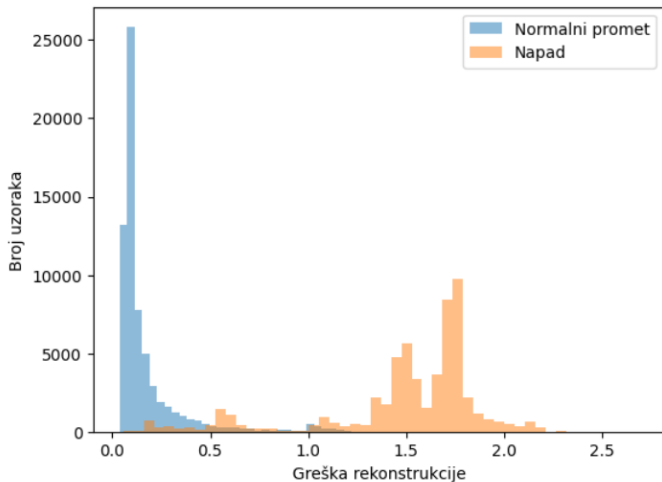
- NSL-KDD: $> 120\,000$ zapisa TCP veza, 125 atributa
- $\sim 50\%$ normalan promet, $\sim 50\%$ napadi
- Treniramo **samo na normalnom prometu**

- NSL-KDD: $> 120\,000$ zapisa TCP veza, 125 atributa
- $\sim 50\%$ normalan promet, $\sim 50\%$ napadi
- Treniramo **samo na normalnom prometu**
- Visoka greška rekonstrukcije $\Rightarrow$ napad

- NSL-KDD: $> 120\,000$ zapisa TCP veza, 125 atributa
- $\sim 50\%$ normalan promet, $\sim 50\%$ napadi
- Treniramo **samo na normalnom prometu**
- Visoka greška rekonstrukcije $\Rightarrow$ napad

Parametri: Slojevi = $[122, 32, 122]$, $\beta = 0.5$, $\rho = 0.2$, $\eta = 0.01$, 3 epohe

Detekcija anomalija — Rezultati



Slika 1: Histogram grešaka rekonstrukcije

- Prag: 0.5 — 2414 napada ispod praga

- Prag: 0.5 — 2414 napada ispod praga
- **Lako detektirati:** neptune — 0/41 214 propušteno
 - Kratke veze, bez podataka, patološka struktura zapisa

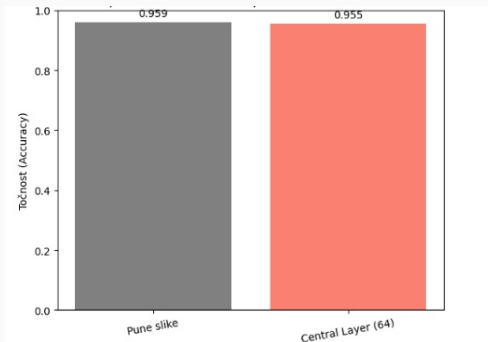
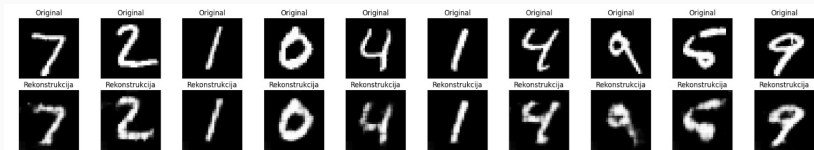
- Prag: 0.5 — 2414 napada ispod praga
- **Lako detektirati:** neptune — 0/41 214 propušteno
 - Kratke veze, bez podataka, patološka struktura zapisa
- **Teško detektirati:** warezmaster (95%), warezclient (85.6%), back (82%)
 - Maliciozni sadržaj van konteksta TCP zapisa
 - Legitimna veza s malicioznim sadržajem

Propušteni napadi po tipu

Tip napada	Prop.	Ukupno	Avg. greška
warezmaster	95.0%	20	0.275
warezclient	85.6%	890	0.329
back	82.0%	956	0.320
multihop	71.4%	7	0.595
ftp_write	50.0%	8	0.591
rootkit	40.0%	10	0.634
buffer_overflow	20.0%	30	0.901
nmap	19.4%	1493	0.759
loadmodule	11.1%	9	0.889
ipsweep	8.4%	3599	0.675
satan	6.5%	3633	1.599

Tip napada	Prop.	Ukupno	Avg. greška
guess_passwd	0.0%	53	1.452
imap	0.0%	11	1.592
land	0.0%	18	1.919
neptune	0.0%	41214	1.683
perl	0.0%	3	1.236
phf	0.0%	4	1.037
pod	0.0%	201	1.190
portsweep	0.0%	2931	1.821
smurf	0.0%	2646	1.445
spy	0.0%	2	0.728
teardrop	0.0%	892	1.262

MNIST uloženi u 64D



MNIST uložen u 2D

